

Bug .

Un juego de cartas donde la partida vive en los navegadores de quienes juegan. **No hay servidor de juego.** Nadie tiene la versión buena del estado: o están todos de acuerdo, o no hay partida.

PRIMERO

Qué es y cómo se juega

DESPUÉS

Cómo está hecho

Y EL GRUESO

Cómo se prueba, y con qué

I El juego

Qué es, con qué cartas se juega y cómo entra alguien que pasa por el stand.

Un UNO con temática de desgracias informáticas

Mismo flujo que el juego de cartas de toda la vida: te reparten **7 cartas**, juegas una que **iguale en color, número o símbolo**, y si no puedes, robas. **Gana quien se queda sin cartas.**

Lo que cambia es el disfraz — y algunas mecánicas propias. Aquí no hay un «+2»: hay un *Update de Windows*. No hay «salta»: se te *fue el WiFi*.

Entre **2 y 10 jugadores**, 30 segundos por turno, y se entra escaneando un QR desde el móvil sin instalar nada.

Los cuatro palos sustituyen a los colores. El arte es propio: las cartas están dibujadas en Inkscape y compiladas a un sprite que viaja dentro del HTML, así que están pintadas desde el primer fotograma.



Código

el palo verde



Hardware

el palo magenta



Internet

el palo violeta



Café

el palo rosa

Las de siempre, con otro nombre

SALTA

Se fue el WiFi

El siguiente pierde el turno.

REVERSA

Ctrl+Z

La ronda cambia de sentido.

+2 / +4

Update de Windows

El siguiente roba y calla.

COMODÍN

Reinicio de Router

Se juega sobre cualquier cosa y eliges palo.

COMODÍN +4

Pantalla Azul (BSOD)

Lo mismo, y el siguiente roba cuatro.

SOLO DOS, Y SIN PALO

Los comodines

Se pintan con las cuatro franjas: es lo que significa «no soy de ningún palo».

Un detalle que costó un rato entender jugando: un comodín en el pozo **no dice a qué hay que igualar**. Por eso la carta gira en 3D y aterriza teñida del palo que eligió quien la tiró.

Cuatro mecánicas que el UNO original no tiene

INTERRUPCIÓN

Copiar y Pegar

Se juega **fuera de tu turno** si tienes la carta gemela de la del pozo. Corta la ronda y el turno salta a ti.

REINICIO

Apagar y volver a prender

Tiras el pozo y pones una carta tuya de base. La partida sigue con ese palo.

BARAJADO

Derrame de Café

Todos pasan su mano al siguiente. Tu buena mano deja de ser tuya.

ATAQUE

Virus Troyano

Eliges víctima y le regalas dos cartas tuyas. Cuidado: puede dejarte sin mano.

Copiar y Pegar es la que más se nota en el diseño distribuido: es una jugada que llega *fuera de turno* y desde otro nodo, así que dos personas pueden intentar cortar a la vez. Quién gana no lo decide un servidor — lo decide el reloj lógico, y sale igual en las tres pantallas.

Del QR a la primera carta, sin instalar nada

- 1 **Uno hace de anfitrión** y levanta la imagen: `docker run -p 7787:7787 sketox/bug`. Un solo puerto.
- 2 Escribe su nombre, pulsa **Crear sala** y aparece un **código de 4 caracteres** y un **QR**.
- 3 Los demás **escanean el QR** desde su móvil. El enlace ya lleva dentro la sala y el punto de encuentro: solo ponen su nombre.
- 4 Con 2 o más en el lobby, el anfitrión pulsa **¡Empezar!** — y quién abre lo decide la semilla, no él.
- 5 Se juega. **M** abre la **Pantalla Maestra** para el proyector.

REQUISITO DE FERIA

«El público debe poder interactuar de forma inmediata, escaneando un QR, sin instalar aplicaciones».

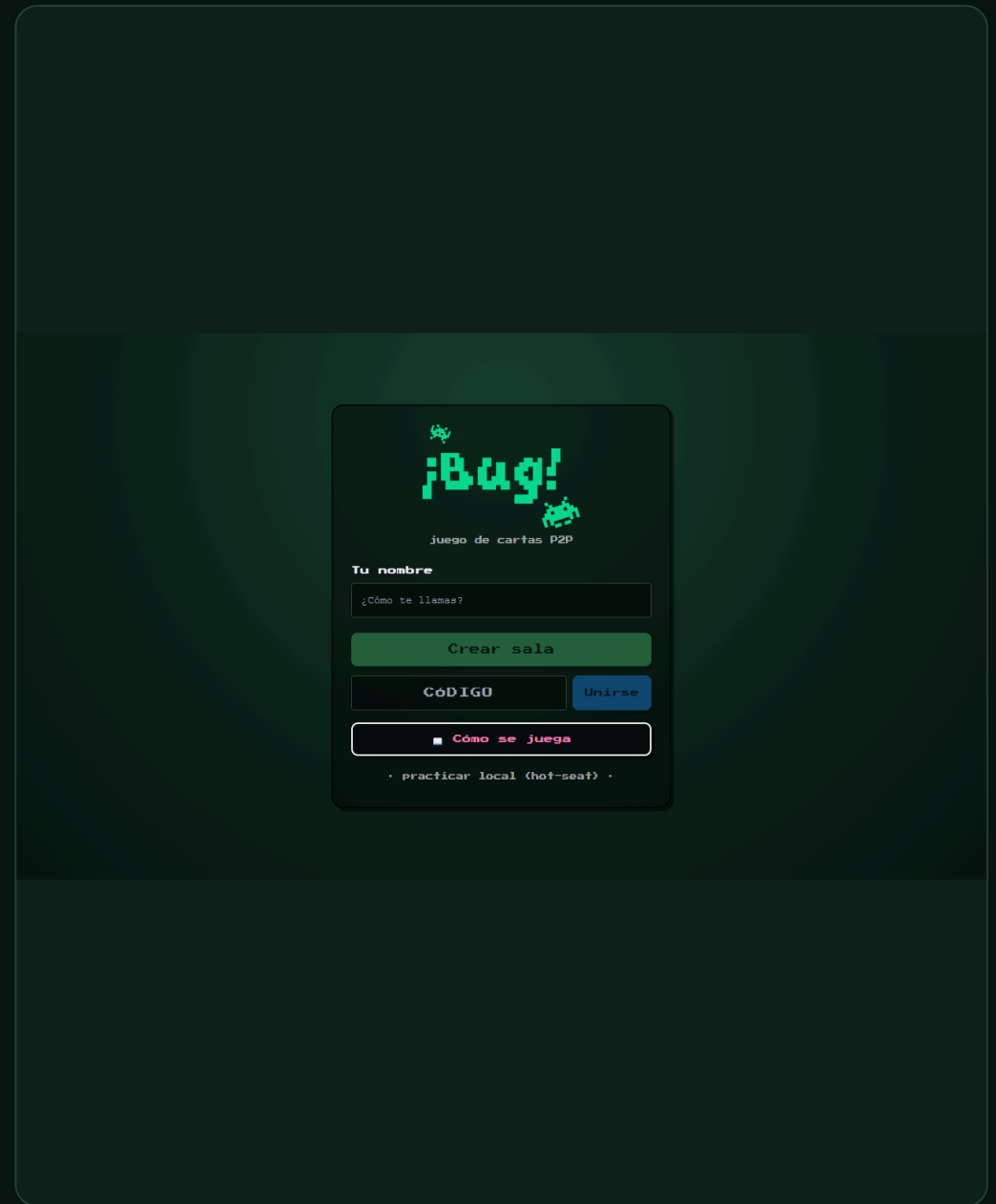
Por eso el QR transporta **sala + señalización**: quien lo escanea no configura nada. Y si el QR apunta a `localhost`, la propia pantalla lo avisa — solo funcionaría en esa máquina.

DURANTE LA PARTIDA

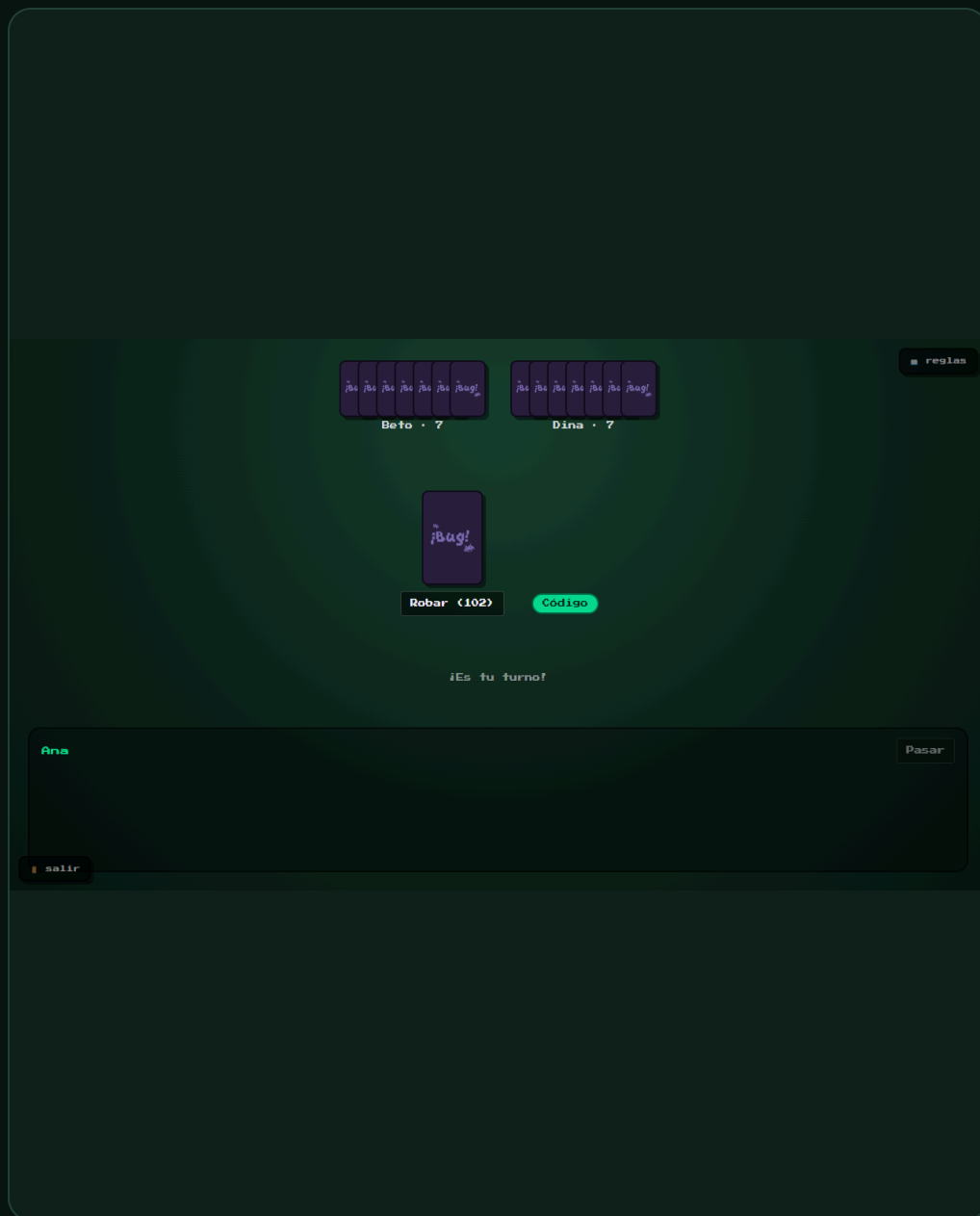
30 s por turno · recargar la página **no te echa**, vuelves con tu mano · si a alguien se le cae la conexión le guardan el sitio y le saltan el turno.



La entrada: nombre, crear o unirse, y practicar en local



«Cómo se juega», con las cartas de verdad: se aprende sin que nadie lo explique



Mesa repartida: siete cartas, mazo, pozo y de quién es el turno



360 px: la anchura desde la que llega casi todo el mundo en la feria

II

Cómo está hecho

Dónde corre el juego de verdad, y qué lo mantiene de acuerdo sin un árbitro.

«Si hay un contenedor encendido, ¿esto no es cliente-servidor?»

El contenedor **reparte el programa y presenta a los jugadores**. El juego **corre entero dentro de cada navegador**.

Cuando alguien abre el enlace, su navegador *se descarga una copia del programa* — como bajarse un PDF. Esa copia se ejecuta en su máquina, con el motor de reglas completo y su propia réplica del estado. El contenedor solo los presenta; a partir de ahí, se aparta.

Es el portero del edificio, no el árbitro de la partida.

La demo que lo demuestra: empezar una partida entre tres y apagar Docker a la mitad. La partida sigue. Solo dejan de poder entrar los nuevos.

	EL CONTENEDOR	TU NAVEGADOR
¿Tiene el motor de reglas?	No	Sí, entero
¿Guarda el estado?	No	Su propia réplica
¿Ve las cartas?	Nunca	Las suyas
¿Decide el turno?	No	El testigo, entre navegadores
¿Sale en la Pantalla Maestra?	No aparece	Sí, como nodo

El WebSocket presenta. El juego va por WebRTC.

ARRANQUE · WEBSOCKET

Encontrarse

Dos navegadores **no pueden encontrarse solos**: no tienen IP pública ni escuchan conexiones entrantes. Alguien tiene que presentarlos. Eso —y solo eso— hace el servidor.

Y hace menos de lo que parece: presenta al que llega con **UN** jugador de la sala. Los demás apretones de manos viajan por la propia malla, dando saltos.

PARTIDA · WEBRTC DATACHANNELS

Jugar

Cada nodo abre un canal con cada otro: **mallá completa**. Las jugadas viajan directas de navegador a navegador, cifradas con DTLS.

Interceptando el WebSocket con un proxy **no se ve ni una carta**: el juego no pasa por ahí.

El bootstrap centralizado es irreducible y lo tienen todos: BitTorrent sus *trackers*, Bitcoin sus *DNS seeds*, IPFS sus nodos de arranque. Lo que se puede elegir es qué pasa después — y aquí, después, el servidor sobra.

Cuatro mecanismos, ningún árbitro

EJE 2 · ORDEN

Relojes de Lamport

Cada jugada lleva su sello lógico. Los eventos concurrentes se desempatan por **peerId**: el mismo orden total en todos, llegue lo que llegue cuando llegue.

EJE 3 · EXCLUSIÓN MUTUA

Testigo de turno

El turno es el recurso crítico. Circula como un testigo con número de secuencia: los anuncios rezagados se descartan solos y nadie puede ceder lo que no tiene.

EJE 3 · CONSISTENCIA

Réplica + huella

Mismo estado inicial (una semilla) + mismo log = mismo estado. Cada nodo publica el **hash** de su estado en cada latido: la convergencia es verificable a simple vista.

EJE 4 · TOLERANCIA

Latidos + Bully

Silencio de 2,5 s → sospechoso; 6 s → caído. Si cae el coordinador se elige otro con el algoritmo del matón, sin que la mesa se pare.

Y si un nodo **diverge** —le faltó un evento, o le sobra— lo detecta comparando su huella con la del líder, pide la partida entera y **la adopta**. Se repara solo, en vivo, sin que nadie toque nada.

Pantalla Maestra cerrar x

LÍDER
Ana

ELECCIÓN
estable

TESTIGO
Beto

SEQ TESTIGO
0

MI LAMPORT
0

EVENTOS EN EL LOG
0

RECHAZADOS
0

CONVERGENCIA
OK ✓

Nodos (3)

● Ana (tú) 👑

vivo dfdadS3a

● Beto 🟡

vivo dfdadS3a

● Dina

vivo dfdadS3a

Sucesos
sin novedad en la malla

reglas

Ana · 7

reglas

Dina · 7

reglas

Ana · 7

reglas

Beto · 7

¡Bug!

Internet

Robar (102)

repartiéndose...

¡EMPIEZAS TÚ!

¡Es tu turno!

⌚ 27s

Beto

Pasar

salir

malla

reglas

Ana · 7

reglas

Dina · 7

¡Bug!

Internet

Robar (102)

repartiéndose...

empieza Beto

Turno de Beto

⌚ 28s

Dina

Pasar

salir

malla

Tecla **M** · quién vive, quién es líder 👑, dónde está el testigo 🟡 y la huella de cada nodo – la columna que decide si hay partida

III

Cómo se prueba

Qué herramienta para qué pregunta — y qué encontró cada una.

No hay servidor al que preguntarle cuál era la verdad

En una aplicación con backend, comprobar el estado es fácil: se le pregunta a la base de datos. Aquí **no hay a quién preguntar**. Hay tres réplicas y ninguna manda.

Por eso la pregunta central no es *¿está bien construido?* (verificación) ni *¿sirve para lo que se pidió?* (validación), sino una tercera:

¿siguen todos de acuerdo?

Y ADEMÁS

Lo que no existe en Node

[RTCPeerConnection](#) no existe fuera de un navegador. La malla de verdad **solo** se puede probar en uno.

Y ADEMÁS

Lo que no se repite

Un evento que llega tarde, un nodo que cae a media elección, dos jugadas en el mismo milisegundo. No se reproducen a mano: hay que **provocarlos**.

227 comprobaciones, cada una en la capa que le toca



Encima de todo queda la validación con personas: la feria. Debajo del todo, el análisis estático, que mira incluso el código que nadie llega a ejecutar.

La pirámide es ancha por abajo —190 unitarias sostienen el motor y los algoritmos— pero también **ancha por arriba**, y eso no es doctrina: es lo que dijeron los defectos.

De los 17 encontrados, **11 eran invisibles para una prueba unitaria**.

Salieron jugando, en un navegador real, atacando el servidor a mano o montando el propio control de calidad.

Para lo que las pruebas no pueden ver

¿Qué pregunta responde?

Una prueba solo puede juzgar el código *que ejecuta*. El análisis estático lee **todo** —incluido lo que nadie llama nunca— y contesta a lo que las pruebas ni se plantean: ¿hay duplicación? ¿qué función se ha vuelto impenetrable? ¿cuánta deuda hemos acumulado? ¿la cobertura sube o baja?

Se analiza el monorepo **como un solo proyecto** y no como cuatro: los cuatro paquetes se despliegan juntos, así que la duplicación entre **engine** y **net** es duplicación de verdad y queremos verla.



QUALITY GATE

Passed

BUGS

0

fiabilidad A

COBERTURA

89 %

100 % en código nuevo

DUPLICACIÓN

1,3 %

umbral < 3 %

⚠ Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. [Learn more](#)

 **SonarQube** community

Projects Issues Rules Quality profiles Quality gates More ▾

🔍 ⓘ Log in

B Bug - juego de carta...
Project

Analysis

 **Overview**

 Issues

 Security hotspots... **Deprecated**

Reporting

 Measures

 Activity

 Quality gate history **New**

Project

<> Code

ⓘ Project information

Bug - juego de cartas P2P descentralizado / Overview

Overview

5.4k Lines of Code · Version 1.7.1 · Last analysis 6 minutes ago



Quality gate ⓘ
Passed 📈

⚠

The last analysis has warnings. [See details](#)

New Code

Overall Code

New Code Since August 8, 2026 Started 6 hours ago

New issues 0 Required = 0	Accepted issues 0 Valid issues that were not fixed	Coverage 100% Required ≥ 80.0% On 2 New Lines to cover.
Duplications 0.0% Required ≤ 3.0% On 376 New Lines.	Security Hotspots 0 Deprecated A	

⚠ **Limited security analysis**

SonarQube Community Build 26.8 · 5 406 líneas · análisis lanzado desde el pipeline

Un número que no se mira no significa nada

1 BUG · ACCESIBILIDAD

El fondo del diálogo de jugada cerraba con ratón y **no con teclado**. Los dos parches evidentes fallaron: un `onKeyDown` en un `div` sin foco no se dispara nunca, y `role="presentation"` cambiaba una queja por dos.

La señal era que el elemento era el equivocado: ahora es un `<dialog>` nativo, que trae hecho `Esc`, foco atrapado y backdrop.

22 AVISOS · TODOS LA MISMA REGLA

`Math.random`. Revisados uno a uno: 16 son partículas de efectos visuales, el resto códigos de sala y semillas **públicos por diseño**.

El único que merecía dudar: `uid()`, que genera el secreto de reconexión. Sus dos ramas reales usan `crypto.0.explorables`.

Y dos incidencias quedaron marcadas como **falso positivo, con la justificación escrita en la herramienta**: el analizador cree que un `<dialog>` no es interactivo. Gestionar hallazgos incluye decidir cuáles no son defectos — lo que no se puede es dejarlos sin mirar.

Para que la calidad deje de depender de que alguien se acuerde

¿Qué pregunta responde?

Todo lo anterior existe solo si **se ejecuta**. Sin integración continua las pruebas se corren cuando uno se acuerda —o sea, cuando ya sospecha algo—. El pipeline por commit convierte 227 comprobaciones en un **hábito automático**: nadie rompe el motor de reglas y se entera tres días después.

Y algo que solo da la CI: **ejecutar en una máquina que no es la tuya**. La mitad de los fallos de este semestre eran «en mi equipo funciona».

RESULTADO

SUCCESS

las 7 etapas

DURACIÓN

7,2 min

DISPARO

por commit

sondeo cada minuto

CONFIGURACIÓN

as code

se reconstruye desde cero

1

Dependencias

2

Tipos

3

Pruebas
+ cobertura

4

Build

5

SonarQube

6

Seguridad

7

Validación
distribuida

Ordenadas por **lo que falla más barato primero**: no tiene sentido gastar cinco minutos construyendo Next.js para descubrir después un error de tipos que se detecta en veinte segundos.

 Jenkins / bug-p2p

Status

</> Changes

Git Polling Log

Builds >

Filter

Today

✓ #5 5:06 PM

August 8, 2026

✗ #4 10:13 PM

✗ #3 10:10 PM

✗ #2 10:08 PM

✗ #1 9:52 PM

✓ bug-p2p

Monorepo de Bug. Dispara con cada commit de la rama principal (sondeo del repo montado en /repo-src).

Last Successful Artifacts

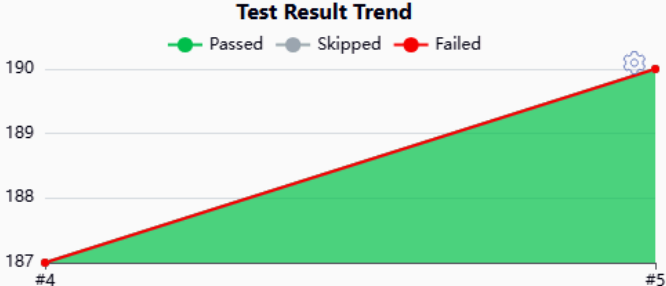
lcov.info	41.41 KiB	view
distribuida-2026-08-10T17-13-34.json	6.97 KiB	view
distribuida-ultimo.json	6.97 KiB	view
seguridad-2026-08-10T17-13-05.json	7.07 KiB	view
seguridad-ultimo.json	7.07 KiB	view

Latest Test Result (no failures)

Permalinks

- Last build (#5), 9 min 11 sec ago
- Last stable build (#5), 9 min 11 sec ago
- Last successful build (#5), 9 min 11 sec ago
- Last failed build (#4), 1 day 19 hr ago
- Last unsuccessful build (#4), 1 day 19 hr ago
- Last completed build (#5), 9 min 11 sec ago

Test Result Trend



Build	Passed	Skipped	Failed
#4	187	0	0
#5	190	0	0

REST API Jenkins 2.568.2

El historial cuenta la verdad: #1 a #4 en rojo son el laboratorio que no arrancaba; #5 es el primer verde, disparado por un commit – 190 pruebas sin fallos y los artefactos archivados

El pipeline llevaba semanas escrito. Nunca se había ejecutado.

- 1 **Jenkins no arrancaba.** El Job DSL dinámico (`scmGit { remotes }`) ya no existe en las versiones de hoy: se llevaba por delante a Configuration-as-Code y el contenedor moría en `Exited (5)`.
- 2 **La corrección no llegaba.** `casc.yaml` vivía dentro de `/var/jenkins_home`, que es un volumen: ahí manda el volumen, no la imagen. Reconstruir no cambiaba nada.
- 3 **El checkout moría.** El plugin de Git no clona rutas locales sin habilitarlo explícitamente.
- 4 **Y moría otra vez, un paso más adentro.** Git rechazaba el repo montado por pertenecer a otro usuario. El mismo error del punto 2 con otro disfraz: `--global` escribe en `$HOME...` que es el volumen.

Y uno silencioso, el peor: el token de Sonar nunca entraba en el contenedor, así que el pipeline habría saltado el análisis en cada build **con el mismo amarillo que si el laboratorio estuviera apagado**. Un fallo disfrazado de comportamiento tolerado no lo investiga nadie.

Porque corre dentro del navegador

El enunciado deja elegir entre Selenium y Cypress. Aquí la elección la decidió una necesidad concreta: para afirmar que **tres réplicas convergieron** hay que leer la huella de estado de cada nodo — un objeto vivo en la memoria de la página.

Cypress se ejecuta **en el mismo bucle de eventos que la aplicación**: entra en `window` y compara esas huellas directamente. Selenium conduce el navegador *desde fuera*, por el protocolo WebDriver, y todo lo que quiera leer tiene que serializarse por ese puente.

Y algo igual de decisivo: los tres nodos son **iframes del mismo origen** en una sola pestaña, y Cypress puede hablarle a cada uno por separado.

MENU.CY.TS

8

entrada, QR, validación del enlace, móvil de 360 px

PARTIDA-LOCAL.CY.TS

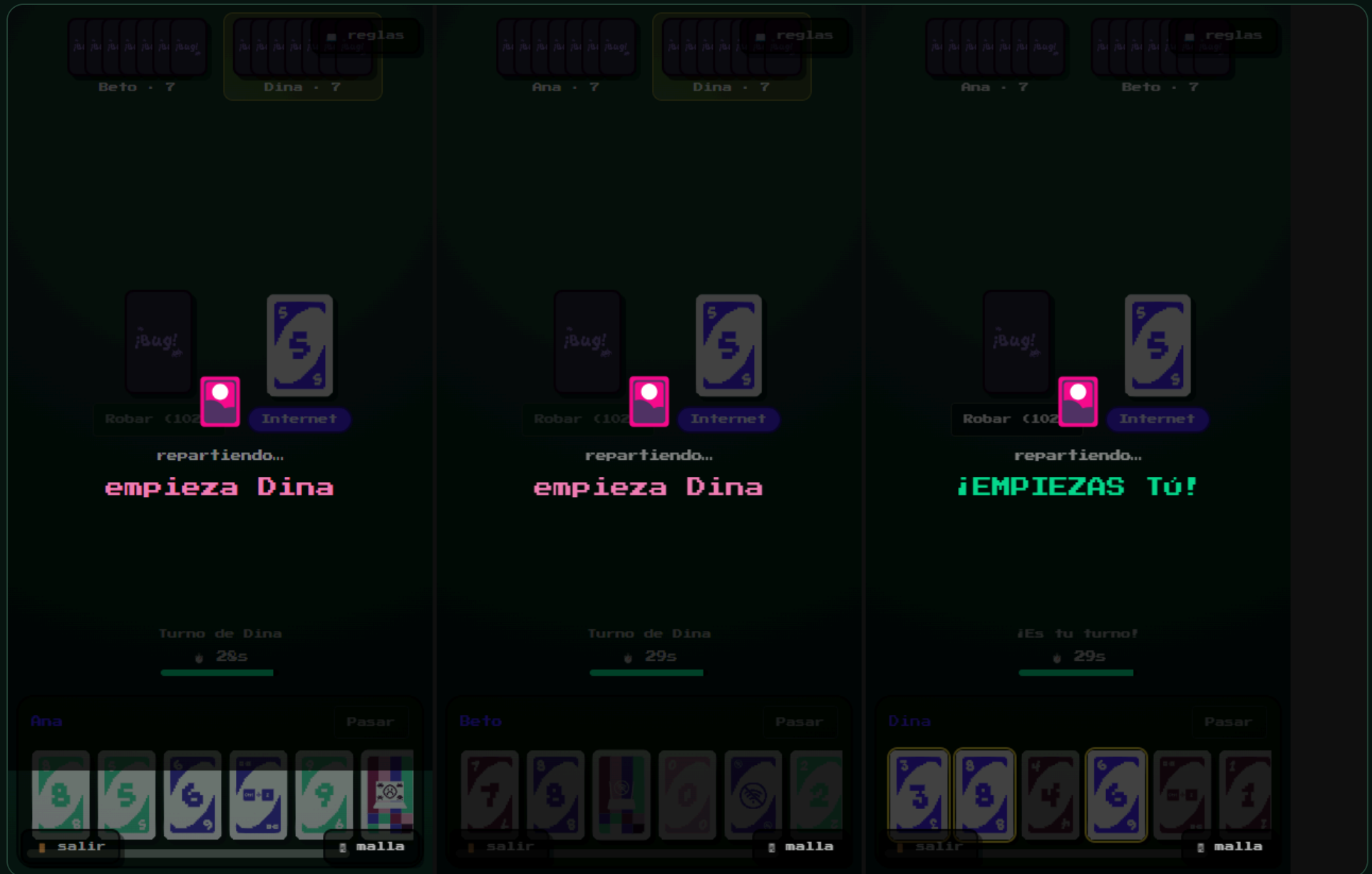
6

reparto, robar, no pasar sin robar, rotación del turno

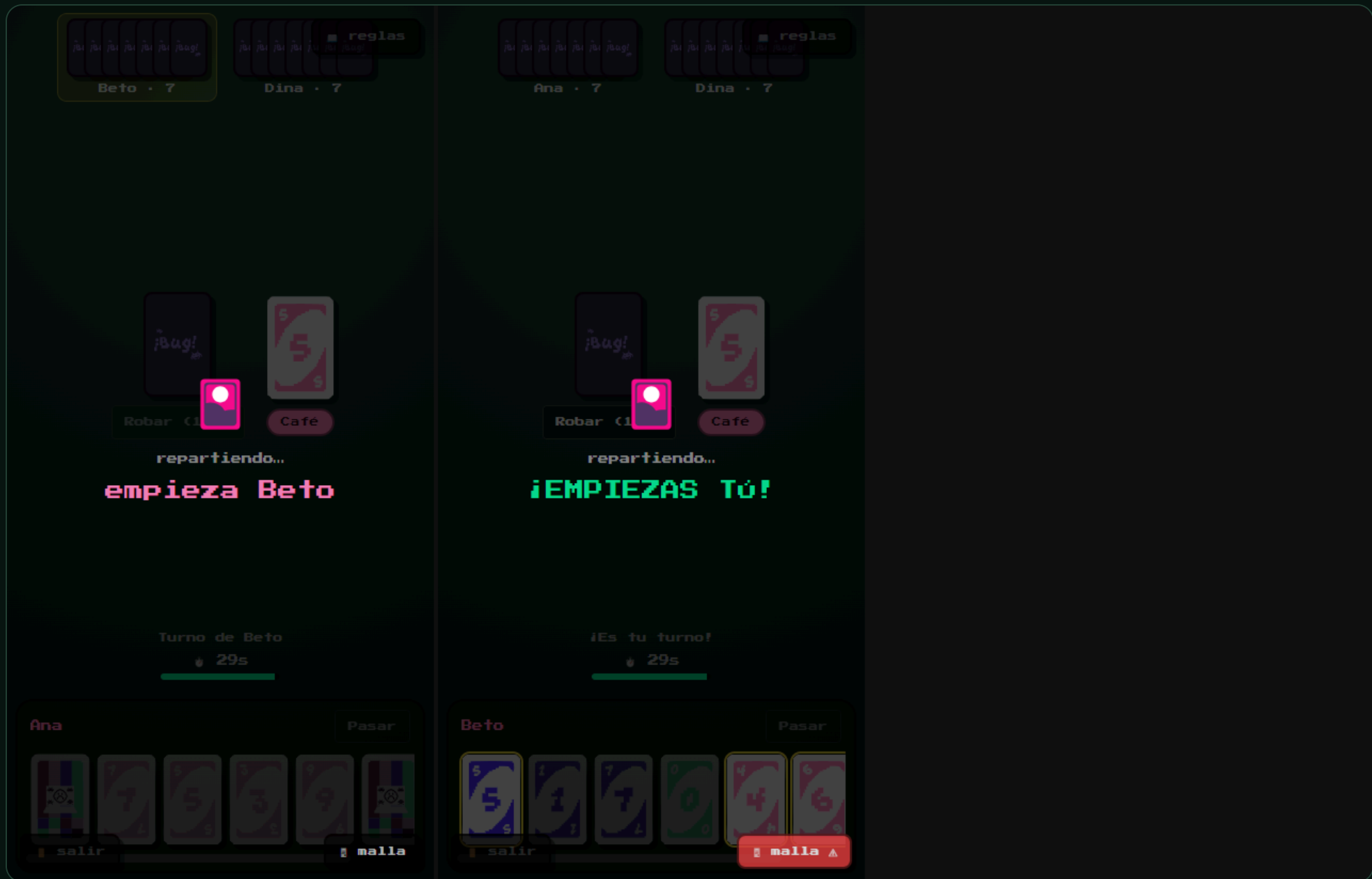
DISTRIBUIDO/MALLA.CY.TS

5

Tres nodos con WebRTC real: lobby, convergencia, jugada compartida, Pantalla Maestra y caída



Ana, Beto y Dina · tres contextos de navegación con su propia `RTCPeerConnection` · la prueba compara sus tres huellas de estado



Se quita el tercer nodo de golpe · los dos que quedan siguen convergiendo entre ellos

Porque un atacante no usa la interfaz

Todas las pruebas anteriores usan el sistema *como está previsto*. Un atacante no: coge un mensaje del protocolo, lo edita a mano y lo reenvía. Para eso hay que sentarse **en medio del tráfico**, y eso es Burp — un proxy de interceptación que además **entiende WebSocket**, que es por donde pasa aquí lo único centralizado.

Con *Repeater* se reenvía un mensaje cambiando un campo; con *Intruder* se repite veinte mil veces. Ninguna prueba unitaria «descubre» un ataque: hay que jugar con el protocolo a mano y ver qué se rompe.

Pero Burp es con lo que se **encuentran** los fallos, no con lo que se **vigilan**. Los once quedaron escritos en un banco que corre en cada construcción: lo que solo vive en la memoria de quien lo encontró vuelve en tres commits.

SEV .	ATAQUE	ANTES	AHORA
Crítica	S3 · secuestro de plaza	×	✓
Crítica	S8 · malformados	×	✓
Alta	S1 · suplantación del emisor	×	✓
Alta	S2 · inyección desde fuera	×	✓
Media	S6 · agotar conexiones	×	✓
Media	S7 · mensaje de 32 MB	×	✓
—	S4-S5-S9-S10-S11 ya defendidos	✓	✓

11/11 bloqueados · 6 vulnerabilidades corregidas

Cuarenta caracteres bastaban para dejar la feria sin señalización

S8 · MENSAJES MALFORMADOS

```
{"t":"introduce","room":"AB12",  
  "peerId":"x","tried":7}
```

Enviado desde la consola del navegador por cualquier jugador de la sala.

El más instructivo, en cambio, es **S1**: el cifrado de WebRTC protege el contenido del canal, **no la identidad de quien lo abrió**. Esa la garantiza la señalización, o no la garantiza nadie.

El servidor hacía `new Set([peerId, ...msg.tried])`. Desparramar un número lanza una excepción, y una excepción dentro de un manejador de eventos de Node sube hasta `uncaughtException` y **mata el proceso**.

Corregido en dos capas: se valida tipo, longitud y forma de todo lo que entra, y el manejador va envuelto en un `try/catch` para el fallo que no se nos ocurrió.

Nadie vende un medidor de **tu** protocolo

Sonar mide código, Cypress conduce navegadores, Burp manipula tráfico. Ninguno sabe qué es «el testigo de turno de Bug» ni qué significa que dos réplicas hayan convergido. Esas propiedades **son del diseño**, así que el instrumento hay que escribirlo.

El banco monta una **red simulada con reloj virtual**: elige latencias entre 5 y 120 ms, duplica entregas, reordena mensajes y provoca caídas encadenadas — cosas que en una red de verdad pasan una vez cada mil partidas y nunca cuando estás mirando.

Y no devuelve un verde: devuelve **números**. El enunciado pide medir latencia y tiempos de respuesta, no aprobar.

ID	PROPIEDAD	MEDICIÓN
D1	Consistencia	3/5/10 nodos · 64 duplicados → misma huella
D2	Orden causal	12 órdenes → 1 sola secuencia
D3	Exclusión mutua	200 cesiones · 0 violaciones
D4	Detección	2 500 ms sospechoso · 6 000 ms caído
D5	Elección (Bully)	acuerdo unánime con caída encadenada
D6	Recuperación	el corrupto no converge sin adoptar
D7	Rendimiento	240 mensajes donde un servidor usaría 360

7/7 propiedades verificadas

Un criterio de aceptación también puede estar mal

D7 exigía que el caso realista convergiera en **menos de 2 segundos de reloj de pared**. Con la máquina cargada, falló — *y las tres réplicas habían convergido perfectamente*.

Un umbral que depende de lo que haya de fondo produce rojos sin defecto detrás. Y un rojo que no significa nada **enseña a ignorar los demás**.

Lo primero que hay que poder creer es la medida. Encontrar esto es tan parte de la V&V como encontrar un bug en el producto — y llega antes.

ANTES · MEDÍA LA MÁQUINA

`msRealista < 2000`

AHORA · MIDE EL ALGORITMO

`aplicaciones por evento ≤ 30×`

Sale idéntico en un portátil cargado y en el agente de CI. Los tiempos se siguen midiendo; ya no deciden.

Solo 6 los habría visto una prueba unitaria

Navegador / jugando		9
Burp Suite		5
Montando la V&V		3

CICLO DE VIDA DE REACT

La sala no se creaba: StrictMode monta, desmonta y vuelve a montar.

ANCHURA EN CSS

El menú desbordaba a 360 px — y así entra casi todo el mundo en la feria.

UN EVENTO A DESTIEMPO

Llegaba antes de existir el motor y se tiraba: divergencia permanente.

No es un argumento contra las pruebas unitarias — son 190 y sostienen el motor y los algoritmos. Es un argumento contra leerlas como si fueran la V&V entera.

Lo que este proceso **no** cubre

- ! **Partida entre redes distintas de verdad.** El código lleva STUN y TURN configurable, pero la prueba necesita dos ubicaciones y no se ha hecho. Es el riesgo abierto más importante para la feria.
 - **Diez jugadores en móviles reales.** Está medido en simulación (D1, D7), no con diez teléfonos.
 - **Accesibilidad completa.** No es requisito del enunciado y no se ha auditado, más allá del diálogo que salió del análisis estático.
 - **Cypress no corre en cada commit.** El agente de CI no tiene navegador; las 19 funcionales se lanzan a mano.
- Un plan de V&V que promete cubrirlo todo no es creíble. Decir dónde no se ha mirado es parte del informe, no una concesión.

Un requisito sin prueba es una intención.

Y una configuración que nunca se ha ejecutado, también. El pipeline llevaba semanas escrito y no arrancaba; el criterio de D7 medía la máquina en vez del algoritmo; el arnés de tres nodos perdía una carrera. Nada de eso se ve leyendo el código.

SONARQUBE

Passed

89 % cobertura · dup. 1,3 %

JENKINS

7/7

etapas, por commit

CYPRESS

19/19

3 nodos, WebRTC real

SEGURIDAD · DISTRIBUIDA

11/11 · 7/7

ataques · propiedades